

NeuralNetwork

Generated by Doxygen 1.15.0

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

ActivationFunction		??
ColumnData		??
Configuration		??
DataPreprocessor		??
LossFunction		??
NeuralNetwork		??

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

NeuralNetworkProjectFoCP/ data_preprocessor.h	??
NeuralNetworkProjectFoCP/ display.h	??
NeuralNetworkProjectFoCP/ file_data_operations.h	??
NeuralNetworkProjectFoCP/ math_functions.h	??
NeuralNetworkProjectFoCP/ neural_network.h	??
NeuralNetworkProjectFoCP/ resource.h	??
UnitTest1/ pch.h	??

Chapter 3

Class Documentation

3.1 ActivationFunction Struct Reference

Public Attributes

- `std::function< std::vector< float >(const std::vector< float > &)>` **func**
- `std::function< std::vector< float >(const std::vector< float > &)>` **derivative**

The documentation for this struct was generated from the following file:

- `NeuralNetworkProjectFoCP/math_functions.h`

3.2 ColumnData Struct Reference

Public Attributes

- `bool` **isNumeric**
- `std::vector< std::string >` **categories**
- `std::unordered_map< std::string, int >` **category_to_index**
- `float` **range** [2]

The documentation for this struct was generated from the following file:

- `NeuralNetworkProjectFoCP/data_preprocessor.h`

3.3 Configuration Struct Reference

Public Attributes

- std::string **initialization_method** = "Xavier"
- int **hidden_layers** = 1
- int **neurons_nb** = 2
- std::vector< std::string > **activations** = {}
- std::string **train_data_path** = "iris.csv"
- std::string **target_column** = "species"
- float **test_fraction** = 0.2f
- int **epochs** = 140
- float **learning_rate** = 0.3f
- std::string **model_save_path** = ""
- std::string **model_load_path** = ""

The documentation for this struct was generated from the following file:

- NeuralNetworkProjectFoCP/main.cpp

3.4 DataPreprocessor Class Reference

Public Member Functions

- void **clear** ()
Clears all the member variables of the [DataPreprocessor](#).
- void **fit** (const std::vector< std::vector< std::string > > &data)
Fits the [DataPreprocessor](#) to the provided dataset.
- std::pair< std::vector< std::vector< float > >, std::vector< std::vector< float > > > **transform_and_extract** (const std::vector< std::vector< std::string > > &data, const std::string &column_to_extract, bool is_there_header=true)
Transforms(one-hot encoding and normalization) and extracts a specific column from the input data.
- void **interpret_extracted_column** (const std::vector< float > &transformed_data) const
Interprets and prints the the actual value - denormalized or decategorized - of the extracted column from the transformed data.
- void **print_state** () const
Prints the current state of the [DataPreprocessor](#).

3.4.1 Member Function Documentation

3.4.1.1 fit()

```
void DataPreprocessor::fit (
    const std::vector< std::vector< std::string > > & data)
```

Fits the [DataPreprocessor](#) to the provided dataset.

Parameters

<i>data</i>	Dataset to fit the preprocessor on - it is expected to have the first row as header.
-------------	--

3.4.1.2 interpret_extracted_column()

```
void DataPreprocessor::interpret_extracted_column (
    const std::vector< float > & transformed_data) const
```

Interprets and prints the the actual value - denormalized or decategorized - of the extracted column from the transformed data.

Parameters

<i>transformed_data</i>	The transformed data of the extracted column (output of transform_and_extract or feedforward from neural_network.cpp)
-------------------------	---

3.4.1.3 transform_and_extract()

```
std::pair< std::vector< std::vector< float > >, std::vector< std::vector< float > > >
DataPreprocessor::transform_and_extract (
    const std::vector< std::vector< std::string > > & data,
    const std::string & column_to_extract,
    bool is_there_header = true)
```

Transforms(one-hot encoding and normalization) and extracts a specific column from the input data.

Parameters

<i>data</i>	Data to be transformed and from which a column is to be extracted
<i>column_to_extract</i>	Name of the column to extract
<i>is_there_header</i>	Indicates if the first row is a header

Returns

Pair where the first element is the transformed data and the second element is the extracted column data

The documentation for this class was generated from the following files:

- NeuralNetworkProjectFoCP/data_preprocessor.h
- NeuralNetworkProjectFoCP/data_preprocessor.cpp

3.5 LossFunction Struct Reference

Public Attributes

- `std::function< std::vector< float >(const std::vector< float > &, const std::vector< float > &)>` **func**
- `std::function< std::vector< float >(const std::vector< float > &, const std::vector< float > &)>` **derivative**

The documentation for this struct was generated from the following file:

- `NeuralNetworkProjectFoCP/math_functions.h`

3.6 NeuralNetwork Class Reference

Public Member Functions

- void [initialize_weights_and_biases](#) (int input_size, int hidden_layers_number, int neurons_per_hidden_layer, int output_size, std::string initialization_method="Xavier", std::vector< std::string > activation_functions={}, int seed=std::random_device())
Initialize weights and biases for the neural network using Xavier uniform initialization, biases are initialized all to zero.
- void [visualize_model](#) () const
Visualize the neural network model structure (weights and biases).
- std::vector< std::vector< float > > [feedforward](#) (const std::vector< std::vector< float > > &input) const
Get a series of outputs of the neural network for a given inputs.
- void [train](#) (const std::vector< std::vector< float > > &inputs, const std::vector< std::vector< float > > &targets, const std::vector< std::vector< float > > &test_inputs, const std::vector< std::vector< float > > &test_targets, int epochs, float learning_rate, const std::string &loss_function_name)
Train the neural network using the provided training data.
- float [test_model](#) (const std::vector< std::vector< float > > &test_inputs, const std::vector< std::vector< float > > &test_targets, const std::string &loss_function_name) const
Test the neural network model using the provided test data.
- void [save_model_to_file](#) (const std::string &filename) const
Load weights from a file. (each line corresponds to a neuron's weights, each weight is separated by a comma, each layer is separated by an empty line).
- void [load_model_from_file](#) (const std::string &filename)
Load weights from a file. (each line corresponds to a neuron's weights, each weight is separated by a comma).
- const std::vector< std::vector< std::vector< float > > > & [get_weights](#) () const
Get the weights of the neural network.
- const std::vector< std::vector< float > > & [get_biases](#) () const
Get the biases of the neural network.
- const std::vector< std::string > & [get_activations](#) () const
Get the activation functions of the neural network.

3.6.1 Member Function Documentation

3.6.1.1 feedforward()

```
std::vector< std::vector< float > > NeuralNetwork::feedforward (
    const std::vector< std::vector< float > > & input) const
```

Get a series of outputs of the neural network for a given inputs.

Parameters

<i>input</i>	The input vector - a 2D vector where each row represents a different input sample.
--------------	--

Returns

The output vector after passing through the network - a 2D vector where each row represents the output for the corresponding input sample.

3.6.1.2 get_activations()

```
const std::vector< std::string > & NeuralNetwork::get_activations () const
```

Get the activation functions of the neural network.

Returns

A constant reference to the activation functions.

3.6.1.3 get_biases()

```
const std::vector< std::vector< float > > & NeuralNetwork::get_biases () const
```

Get the biases of the neural network.

Returns

A constant reference to the biases.

3.6.1.4 get_weights()

```
const std::vector< std::vector< std::vector< float > > > & NeuralNetwork::get_weights ()
const
```

Get the weights of the neural network.

Returns

A constant reference to the weights.

3.6.1.5 initialize_weights_and_biases()

```
void NeuralNetwork::initialize_weights_and_biases (
    int input_size,
    int hidden_layers_number,
    int neurons_per_hidden_layer,
    int output_size,
    std::string initialization_method = "Xavier",
    std::vector< std::string > activation_functions_passed = {},
    int seed = std::random_device()())
```

Initialize weights and biases for the neural network using Xavier uniform initialization, biases are initialized all to zero.

Parameters

<i>input_size</i>	The size of the input layer.
<i>hidden_layers_number</i>	The number of hidden layers.
<i>neurons_per_hidden_layer</i>	The number of neurons in each hidden layer.
<i>output_size</i>	The size of the output layer.
<i>initialization_method</i>	The method used for weight initialization (either "Xavier" or "He").
<i>activation_functions_passed</i>	Activation functions for each layer - should match the number: <code>hidden_layers_number + 1</code> (output layer). If invalid or empty, defaults are sigmoids
<i>seed</i>	The random seed for weight initialization.

3.6.1.6 load_model_from_file()

```
void NeuralNetwork::load_model_from_file (
    const std::string & filename)
```

Load weights from a file. (each line corresponds to a neuron's weights, each weight is separated by a comma).

Parameters

<i>filename</i>	The name of the file to load the weights from.
-----------------	--

3.6.1.7 save_model_to_file()

```
void NeuralNetwork::save_model_to_file (
    const std::string & filename) const
```

Load weights from a file. (each line corresponds to a neuron's weights, each weight is separated by a comma, each layer is separated by an empty line).

Parameters

<i>filename</i>	The name of the file to load the weights from.
-----------------	--

3.6.1.8 test_model()

```
float NeuralNetwork::test_model (
    const std::vector< std::vector< float > > & test_inputs,
    const std::vector< std::vector< float > > & test_targets,
    const std::string & loss_function_name) const
```

Test the neural network model using the provided test data.

Parameters

<i>test_inputs</i>	The input data for testing.
<i>test_targets</i>	The target output data for testing.
<i>loss_function_name</i>	The name of the loss function to use - right now only "MSE".

Returns

The computed loss value at the end.

3.6.1.9 train()

```
void NeuralNetwork::train (
    const std::vector< std::vector< float > > & inputs,
    const std::vector< std::vector< float > > & targets,
    const std::vector< std::vector< float > > & test_inputs,
    const std::vector< std::vector< float > > & test_targets,
    int epochs,
    float learning_rate,
    const std::string & loss_function_name)
```

Train the neural network using the provided training data.

Parameters

<i>inputs</i>	The input data for training - each row represents a training sample. It is expected to be shuffled.
<i>targets</i>	The target output data for training - each row represents the target output for the corresponding training sample.
<i>test_inputs</i>	The input data for testing during training - each row represents a different test sample.
<i>test_targets</i>	The target output data for testing during training - each row represents the target output for the corresponding test sample.
<i>epochs</i>	The number of epochs to train for.
<i>learning_rate</i>	The learning rate for weight updates.
<i>loss_function_name</i>	The name of the loss function to use - right now only "MSE".

The documentation for this class was generated from the following files:

- NeuralNetworkProjectFoCP/neural_network.h
- NeuralNetworkProjectFoCP/neural_network.cpp

Chapter 4

File Documentation

4.1 data_preprocessor.h

```
00001 #ifndef DATAPREPROCESSOR_H
00002 #define DATAPREPROCESSOR_H
00003
00004 #include <vector>
00005 #include <string>
00006 #include <unordered_map>
00007
00008 // Structure to hold information about each column
00009 struct ColumnData {
00010     bool isNumeric;
00011     std::vector<std::string> categories; // All unique categories for categorical columns
00012     std::unordered_map<std::string, int> category_to_index;
00013     float range[2]; // Min and Max for numeric columns
00014 };
00015
00016 class DataPreprocessor {
00017 private:
00018     std::vector<ColumnData> columns; // Column index -> column_data
00019     std::unordered_map<std::string, int> column_order;
00020     std::string extracted_column_name;
00021     bool is_fitted = false;
00022
00023 public:
00024     void clear();
00025
00026     void fit(const std::vector<std::vector<std::string>& data);
00027
00028     std::pair<std::vector<std::vector<float>, std::vector<std::vector<float>>>
    transform_and_extract(const std::vector<std::vector<std::string>& data, const std::string&
    column_to_extract, bool is_there_header = true);
00029
00030     void interpret_extracted_column(const std::vector<float>& transformed_data) const;
00031     void print_state() const;
00032 };
00033
00034 #endif // DATAPREPROCESSOR_H
```

4.2 display.h

```
00001 #pragma once
00002 #ifndef DISPLAY_H
00003 #define DISPLAY_H
00004
00005 #include <string>
00006
00007 // Prints a main header
00008 void print_header(const std::string& text);
00009
00010
00011 // Prints a separator line (e.g. "-----")
00012 void print_separator(char sep = '-', int width = 60);
00013
00014 void print_vector(const std::vector<float>& vec);
```

```

00015
00016
00024 void print_feedforward_output(const std::vector<std::vector<float>>& feedforward_input,
00025                                const std::vector<std::vector<float>>& target_output,
00026                                const std::vector<std::vector<float>>& feedforward_output,
00027                                const size_t sample_count = 5);
00028 #endif // DISPLAY_H

```

4.3 file_data_operations.h

```

00001 #pragma once
00002 #ifndef FILE_DATA_OPERATIONS_H
00003 #define FILE_DATA_OPERATIONS_H
00004
00005 #include <vector>
00006 #include <string>
00007 #include <fstream>
00008
00009 //Parsing functions
00010 std::vector<std::string> get_strings_from_line(const std::string& line);
00011 std::vector<float> get_floats_from_line(const std::string& line);
00012
00013 //Loading/saving to file functions
00014 std::vector<std::vector<std::string>> parseCSV(const std::string& filename);
00015
00016 // Overload for 1D vectors
00017 void save_vector_to_file(const std::string& filename,
00018                         const std::vector<float>& vec,
00019                         char delimiter = ',');
00020 void save_vector_to_file(const std::string& filename, const std::vector<std::string>& vec, char
00021                         delimiter = ',');
00022 // Overload for 2D vectors
00023 void save_vector_to_file(const std::string& filename,
00024                         const std::vector<std::vector<float>>& vec,
00025                         char delimiter = ',');
00026 // Overload for 3D vectors
00027 void save_vector_to_file(const std::string& filename, const
00028                         std::vector<std::vector<std::vector<float>>>& vec, char delimiter = ',');
00029
00030 void load_vector_from_file(const std::string& filename, std::vector<std::string>& vec, char delimiter
00031                         = ',');
00032 void load_vector_from_file(const std::string& filename, std::vector<std::vector<float>>& vec, char
00033                         delimiter = ',');
00034 void load_vector_from_file(const std::string& filename, std::vector<std::vector<std::vector<float>>>&
00035                         vec, char delimiter = ',');
00036
00037 int get_index(const std::vector<std::string>& vec, const std::string& value);
00038 #endif

```

4.4 math_functions.h

```

00001 #pragma once
00002
00003
00004 #include <vector>
00005 #include <string>
00006 #include <map>
00007 #include <functional>
00008
00009 std::vector<float> relu(const std::vector<float>& x);
00010 std::vector<float> relu_derivative(const std::vector<float>& x);
00011
00012 std::vector<float> sigmoid(const std::vector<float>& x);
00013 std::vector<float> sigmoid_derivative(const std::vector<float>& x);
00014
00015 std::vector<float> mtanh(const std::vector<float>& x);
00016 std::vector<float> mtanh_derivative(const std::vector<float>& x);
00017
00018 std::vector<float> mean_squared_error(const std::vector<float>& predicted, const std::vector<float>&
00019                                     actual);
00020 std::vector<float> mean_squared_error_derivative(const std::vector<float>& predicted, const
00021                                                  std::vector<float>& actual);
00022
00023 float mean_squared_error_scalar(const float& predicted, const float& actual);
00024
00025 struct ActivationFunction {
00026     std::function<std::vector<float>>(const std::vector<float>&)> func;

```



```

00025     std::function<std::vector<float>(const std::vector<float>&)> derivative;
00026 };
00027
00028 struct LossFunction {
00029     std::function<std::vector<float>(const std::vector<float>&, const std::vector<float>&)> func;
00030     std::function<std::vector<float>(const std::vector<float>&, const std::vector<float>&)>
    derivative;
00031 };
00032
00033 extern const std::map<std::string, ActivationFunction> activation_map;
00034 extern const std::map<std::string, LossFunction> loss_function_map;
00035
00036
00037
00038 // Utility function to sum elements of a vector
00039 float sum_vector(const std::vector<float>& vec);

```

4.5 neural_network.h

```

00001 #ifndef NEURAL_NETWORK_H
00002 #define NEURAL_NETWORK_H
00003
00004 #include <vector>
00005 #include <string>
00006 #include <random>
00007
00008 class NeuralNetwork {
00009 public:
00010     NeuralNetwork();
00011     void initialize_weights_and_biases(int input_size, int hidden_layers_number, int
    neurons_per_hidden_layer, int output_size, std::string initialization_method = "Xavier",
00012     std::vector<std::string> activation_functions = {},
00013     int seed = std::random_device()());
00014     void visualize_model() const;
00015     std::vector<std::vector<float>> feedforward(const std::vector<std::vector<float>>& input) const;
00016
00017     void train(const std::vector<std::vector<float>>& inputs,
00018     const std::vector<std::vector<float>>& targets,
00019     const std::vector<std::vector<float>>& test_inputs,
00020     const std::vector<std::vector<float>>& test_targets,
00021     int epochs,
00022     float learning_rate,
00023     const std::string& loss_function_name);
00024
00025     float test_model(const std::vector<std::vector<float>>& test_inputs,
00026     const std::vector<std::vector<float>>& test_targets,
00027     const std::string& loss_function_name) const;
00028     void save_model_to_file(const std::string& filename) const;
00029     void load_model_from_file(const std::string& filename);
00030
00031     // Getters (mainly for testing)
00032     const std::vector<std::vector<std::vector<float>>>& get_weights() const;
00033     const std::vector<std::vector<float>>& get_biases() const;
00034     const std::vector<std::string>& get_activations() const;
00035
00036 private:
00037     std::vector<std::vector<std::vector<float>>> weights;
00038     std::vector<std::vector<float>> biases;
00039     std::vector<std::string> activations;
00040
00041     bool is_model_valid = false;
00042     void validate_model_structure();
00043
00044 };
00045 #endif

```

4.6 resource.h

```

00001 //{NO_DEPENDENCIES}
00002 // Plik doczany wygenerowany przez rodowisko Microsoft Visual C++.
00003 // Uywany przez: NeuralNetworkProjectFoCP.rc
00004 //
00005 #define IDR_MENU1 101
00006 #define ID_HEJ_HWEH 40001
00007
00008 // Next default values for new objects
00009 //
00010 #ifdef APSTUDIO_INVOKED

```

```
00011 #ifndef APSTUDIO_READONLY_SYMBOLS
00012 #define _APS_NEXT_RESOURCE_VALUE        102
00013 #define _APS_NEXT_COMMAND_VALUE         40002
00014 #define _APS_NEXT_CONTROL_VALUE         1001
00015 #define _APS_NEXT_SYMED_VALUE           101
00016 #endif
00017 #endif
```

4.7 pch.h

```
00001 // pch.h: wstępnie skompilowany plik nagłówka.
00002 // Wymienione poniżej pliki są kompilowane tylko raz, co poprawia wydajność kompilacji dla przyszłych
    kompilacji.
00003 // Ma to także wpływ na wydajność funkcji IntelliSense, w tym uzupełnianie kodu i wiele funkcji
    przeglądania kodu.
00004 // Jednak WSZYSTKIE wymienione tutaj pliki będą ponownie kompilowane, jeśli którykolwiek z nich
    zostanie zaktualizowany między kompilacjami.
00005 // Nie dodawaj tutaj plików, które będziesz często aktualizować (obniża to korzystny wpływ na
    wydajność).
00006
00007 #ifndef PCH_H
00008 #define PCH_H
00009
00010 // w tym miejscu dodaj nagłówki, które mają być wstępnie kompilowane
00011
00012 #endif //PCH_H
```